

Twenty tabulated lower bounds for sorting networks rest on an invalid proof: an audit of Van Voorhis (1972), and a partial repair

Abstract

The lower bounds on sorting-network size tabulated for $N \geq 13$ in the most consulted public table rest on a two-value pruning theorem of Van Voorhis (1972) whose published proof is invalid. Its equations (5) and (6) are false: we exhibit a size-optimal three-comparator network on which (5) overstates the quantity it claims to compute and (6)’s Kraft sum equals $5/4$, and equation (7), the only route from them to the result, rests on them alone. The false premise is that the MAX subnetwork of an N -sorter consists of $N - 1$ comparators, which ignores *pass-through* comparators, present in 61.0% of an audited population. The same chain reproduces the published table at every $N = 13, \dots, 32$, so twenty entries depend on it, and the two most consulted public tables contradict each other at $N = 13, \dots, 20$. We prove the theorem for two classes of networks, covering 22.4% of an audited corpus; half of that corpus has no proof by any route given here. The best published bound, $S(13) \geq 43$, is untouched.

1 Introduction

A *comparator* (a, b) with $a < b$ on n channels replaces the values on channels a and b by their minimum on a and their maximum on b . An n -*sorter* is a sequence of comparators that sorts every input, and $S(n)$ is the least size of one — the Bose–Nelson sorting problem [2]. $S(n)$ is known exactly only for $n \leq 12$:

n	1	2	3	4	5	6	7	8	9	10	11	12	13
$S(n)$	0	1	3	5	9	12	16	19	25	29	35	39	?

Values through $n = 8$ are due to Floyd and Knuth [12], $S(9)$ and $S(10)$ to Codish, Cruz-Filipe, Frank and Schneider-Kamp [5], and $S(11)$ and $S(12)$ to Harder [13], each of the last four by exhaustive search with a machine-checked or independently checked certificate [7, 8, 14]. At $n = 13$ the best construction remains Juillé’s 45-comparator network [15].

Two lower bounds for $S(13)$ are in circulation, and they differ. The **one-value bound** $S(n) \geq S(n - 1) + \lceil \log_2 n \rceil$ gives $S(13) \geq 43$. It is Van Voorhis’s *IEEE Transactions* note [28], Knuth’s exercise 5.3.4–42 with a three-line answer [17], and Harder’s Lemma 17, proved there in full [13]. This is the best *published* lower bound, and nothing here disturbs it (Section 6). The **two-value bound** $S(N) \geq S(N - 2) + P(2, N)$ with $P(2, N) \geq \lceil \log_2 F(N) \rceil$ for an explicitly computable F is the content of Van Voorhis’s chapter in *Complexity of Computer Computations* [27]; with $F(13) = 392$ and the modern value $S(11) = 35$ it yields 44. The chapter’s own Table 1 printed $L(13) = 42$, because in 1972 the best available $S(11)$ was 33, so the 44 is a modern recombination. It entered the public record on 2025-04-21 through a changelog line in Dobbelaere’s table [10]; Wikipedia’s table still prints 43 [30].

We audit the chapter’s proof of the two-value bound and find it invalid. Two of its structural steps are false, and the step that carries the whole conclusion is derived from those two and from nothing else. The error is a single premise, stated once on p. 121 and never revisited: that the MAX subnetwork of an N -sorter consists of exactly $N - 1$ comparators. It does not, whenever the network contains what we call a *pass-through* comparator, and pass-throughs are ordinary: 61.0% of an audited population of sorters has one. The counterexample is a three-comparator network on three channels, and can be checked by hand. We write this out because the argument is fifty-four years old, has never been re-derived, re-proved or machine-checked, and now supports a published table that people use.

Because the same inequality chain is what produces the published table above $N = 12$, the audit does not bear on $S(13)$ alone. We reconstruct the chain to $N = 32$ and it reproduces the published

lower bounds at every one of $N = 13, \dots, 32$, exactly, twenty entries out of twenty — the first written derivation of a column published with no proof and no citation. Every one of those twenty strictly exceeds the one-value floor, so every one depends on the equation the audit breaks. The public record is meanwhile three-way inconsistent: Wikipedia prints the one-value column, Dobbelaere prints the two-value column, and OEIS A003075 [19], credited in Dobbelaere’s column header, lists no lower bound at all for $N \geq 13$. This work says which of the two columns is currently defensible.

We then repair what can be repaired. The missing step is an antichain argument; we supply it for two classes of networks, prove a complementary bound at the opposite extreme, and close one family of would-be repairs structurally. The proved classes cover 22.4 % of the audited corpus, and the uncovered zone is intermediate — the inequality is provable at the pass-through-poor end and again at the pass-through-rich end, with the open problem strictly in between. The repair yields no new numerical bound at any n .

Related work. *Exact values.* The four largest known values of $S(n)$ come from exhaustive search with symmetry breaking [5, 13], formalized in Coq for $n = 9$ [7, 8] and in Isabelle/HOL for $n = 11, 12$ [13]; two-layer prefix generation [6] and depth-oriented pruning [3, 11, 18] address parallel depth, which does not constrain size. *Lower bounds.* Apart from the information-theoretic $\lceil \log_2 n! \rceil$ and the asymptotic $\Omega(n \log n)$ of [16], every published size lower bound above $n = 12$ is a chain from an exact value by one of the two Van Voorhis inequalities. *Engagement with the chapter.* OpenAlex records 18 citing works for [27] as of 2026-08-22, two of them since 2024 [29, 9]; none engages with equations (5)–(9), and the modern ones — including the reinforcement-learning constructions of [29, 4] at exactly these widths — cite the chapter for the one-value bound. A recent lower bound for *stable standard* networks [24] concerns a different quantity. We found no erratum, correction, gap note, reproof or formalization of the two-value argument, and no correction-genre paper of any kind in the sorting-network literature.

Artifact, and what follows. Five standard-library Python programs accompany this paper; they are deterministic, read-only and dependency-free, and re-run every number below in under five minutes in total. The two that carry the audit were written independently: one alongside the first reading of the chapter, the other afterwards from the page images by a reader whose brief was to defend the chapter. Every numerical claim below names the script that recomputes it; check-level indexes, full proofs, refuted routes and known script defects are in the artifact repository [21]. Section 2 fixes vocabulary; Section 3 is the audit; Section 4 is the literature record and the twenty entries; Section 5 is the partial repair; Section 6 says what is not disturbed, including the unconditional structure of a hypothetical minimum-size 13-sorter; Section 7 states what we trust.

2 Preliminaries

We match the verifier code, because several distinctions below are exactly what the 1972 argument elides. Fix an N -sorter T ; outputs ascend, so o_N is the last channel. A *lead* is a wire segment: a channel’s input segment, or a comparator’s output segment.

Definition 1 (max-paths, branch tree, pass-throughs). *Give input channel k a value strictly greater than all others. It leaves every comparator it enters on the high output, tracing a unique max-path p_k to o_N ; put $\text{MAX}(T) = \bigcup_k p_k$. Call a lead red if some one-hot max scenario puts the maximum on it, and blue otherwise. A traversed comparator is a branch node if both its input leads are red, and a pass-through if exactly one is. The branch nodes form the branch tree B : N leaves, $N - 1$ internal nodes, rooted at the last branch node before o_N . T is clean if it has no pass-through.*

For a branch node c with branch-tree subtrees of heights $\text{lp}(L(c))$ and $\text{lp}(R(c))$ and depth $\text{depth}_B(c)$, put

$$a(c) := \text{lp}(L(c)) + \text{lp}(R(c)) + \text{depth}_B(c) + 1, \quad f(B) := \sum_c 2^{a(c)}.$$

Then f satisfies the chapter's Theorem 1 recursion, and $F(N) = \min_B f(B)$ reproduces its Table 1 exactly for $N \leq 16$ under two independent implementations (`verify_huffman2.py`, `verify_kraft_dispute.py`).

For channels $x \neq y$, run the scenario *maximum at x , second maximum at y* and let $W(x, y)$ be the set of comparators either value traverses. Removing them and rewiring leaves an $(N - 2)$ -sorter, so with $p(2, T) := \max_{x \neq y} |W(x, y)|$ we have $|T| \geq S(N - 2) + p(2, T)$: the chapter's equation (2), not in dispute. Write $W^*(c) = \max\{|W(x, y)| : \text{LCA}_B(x, y) = c\}$, so $p(2, T) = \max_c W^*(c)$. The disputed claim, the chapter's equation (8), is

$$p(2, T) \geq \lceil \log_2 f(B) \rceil. \quad (8)$$

Definition 2 (escapes; the hypotheses (C) and (D)). *In the scenario (x, y) the two values first meet at $c = \text{LCA}_B(x, y)$. T is escape-free if the second maximum then occupies no red lead at or after c 's low output; equivalently [21], if it never re-meets the maximum at a branch node. Write $nq(c)$ for the number of comparators the second maximum traverses strictly after c , $ov(c)$ for those of them the maximum also traverses, and $\gamma(c)$ for the number of pass-throughs on the maximum's stem from c to o_N . The two hypotheses isolated below are*

$$(C) \quad \sum_c 2^{-nq(c)} \leq 1, \quad (D) \quad \gamma(c) \geq ov(c) \text{ for every branch node } c.$$

Every clean network is escape-free and the converse fails; the gap is not cosmetic, since Batchier's 12-sorter [1] has a pass-through yet is escape-free.

3 The audit

3.1 The chain, isolated

The section of [27] bounding $P(2, N)$ opens with a premise, attributed to the *Transactions* note [28]:

"It has been observed ... that the MAX subnetwork of any N -sorter is a binary tree with N leaves (the input leads) and $N - 1$ branch nodes (the comparators) rooted at o_N ." — p. 121

"Let T be any N -sorter. The MAX subnetwork of T , $\text{MAX}(T)$, includes $N - 1$ comparators, which we label $c_1, c_2, \dots, c_{N-1}$." — p. 121

For a branch node c_j , $\text{nc}(c_j)$ counts the comparators from the two inputs of its subtrees to c_j and on to o_N , comparator c_j itself counted once; q_j is the second maximum's onward path to o_{N-1} , and the chapter asserts that the paths q_j "together form a binary tree rooted at o_{N-1} , which we call the MAX2 subnetwork" (p. 122). Then

"... the two paths ... together include $\text{nc}(C_j) + \text{nc}(q_j)$ comparators. Therefore, $p(2, T) = \max_j [\text{nc}(C_j) + \text{nc}(q_j)]$." — p. 122, eq. (5)

"MAX2(T) is a binary tree, so the path lengths $\text{nc}(q_j)$ satisfy $\sum_j 2^{-\text{nc}(q_j)} = 1$." — p. 124, eq. (6)

From (5) and (6) alone the chapter derives (7), $\sum_j 2^{-[p(2, T) - \text{nc}(C_j)]} \leq 1$, hence (8), $p(2, T) \geq \lceil \log_2 f(\text{MAX}(T)) \rceil$, hence $P(2, N) \geq \lceil \log_2 F(N) \rceil$ and the tabulated value $35 + 9$ at $N = 13$. There are no side conditions anywhere in the section: no standard-form restriction, no normalization, no minimality hypothesis, no restriction on N . Step (7) needs only the inequalities $\text{nc}(q_j) \leq p(2, T) - \text{nc}(C_j)$ and $\sum_j 2^{-\text{nc}(q_j)} \leq 1$; the failures below are in the fatal direction anyway.

3.2 The counterexample

Proposition 3 (the 3-sorter refutes (5) and (6)). *Let $T_1 = [(0, 1), (0, 2), (1, 2)]$, the Batchier 3-sorter, with $|T_1| = 3 = S(3)$. Then $|\text{MAX}(T_1)| = 3$, not $N - 1 = 2$; equation (5) returns 4 for a quantity whose value is 3; equation (6)'s sum is $5/4$; and under the literal reading of the chapter's equation (9), equation (8) itself is false on T_1 .*

Proof. Write c_0, c_1, c_2 for the three comparators in order. Every one of them is traversed by a max-path, so $\text{MAX}(T_1) = \{c_0, c_1, c_2\}$; but $c_1 = (0, 2)$ is a pass-through, since the maximum enters and leaves it on channel 2 and no max-path arrives on channel 0 there. The branch nodes are $\{c_0, c_2\}$ — two of them, as the chapter says. What fails is their identification with the comparators of $\text{MAX}(T)$.

Take c_0 , with the input pair (channel 0, channel 1). The maximum's path is c_0, c_2 and the second maximum's is c_0, c_1, c_2 , so $\text{nc}(c_0) = \text{nc}(q_0) = 2$ and (5) asserts that the two paths together include 4 comparators. Their union is $\{c_0, c_1, c_2\}$: *three* — after separating at c_0 the two values *meet again* at c_2 . Since $|T_1| = 3$, we have $p(2, T_1) = 3$, confirmed by brute force. So (5) *exceeds* $p(2, T)$, which is exactly the direction that invalidates (7).

The claimed MAX2 leaves are the low output leads of c_0 and c_2 , at depths 2 and 0 — the low output of c_2 is o_{N-1} — giving $\sum_j 2^{-\text{nc}(q_j)} = 5/4$. The cause is structural: q_0 passes through c_2 's low output lead, so the claimed leaves are not an antichain and $\text{MAX2}(T_1)$ is not a tree. Finally, with nc read literally as defined on p. 122, $f = 2^2 + 2^3 = 12$ and (8) asserts $p(2, T_1) \geq 4 > 3$; with f computed by the chapter's Theorem 1 on the branch tree, $f = 8$ and (8) holds with equality. $\square$

Both audit verifiers recompute this independently, and it is checkable by hand. The *tree* reading of nc is forced by the chapter's Fig. 5, whose printed values are exactly the branch-tree quantities, and it is the reading under which the chapter's conclusion is empirically unrefuted; it rescues neither (5) nor (6). The exact replacement for (5) is $|W(x, y)| = \text{nc}(c) + \text{nc}(q_c) - \text{ov}(c)$, and equation (5) is the claim that $\text{ov}(c) = 0$.

3.3 The failure is common, and not confined to slack

Over 387 constructed sorters (bubble, insertion, odd-even transposition and Batcher for $n = 3, \dots, 10$; thinned random prefixes; random channel relabellings), each verified to be a sorter by the 0/1 principle (`verify_kraft_dispute.py`, seed 20260818):

statement	count	share
$\text{MAX}(T)$ has a pass-through	236	61.0%
some q_j re-meets the max-path after c_j	233	60.2%
eq. (5) right-hand side strictly exceeds $p(2, T)$ (fatal)	172	44.4%
eq. (6) sum > 1 (fatal)	164	42.4%
eq. (8) fails, literal nc	102	26.4%
eq. (8) fails, tree nc	0	0.0%
conclusion $p(2, T) \geq \lceil \log_2 F(n) \rceil$ fails	0	0.0%
tight conclusion <i>and</i> eq. (6) broken	105	27.1%

The last row answers the natural dismissal, that this is a technicality biting only where the bound has slack. The exhaustive check is sharper. Over the complete set of 149,040 size-optimal 5-sorters, equation (8) is tight on every single one, and equation (6)'s sum exceeds 1 on 43.9% of them (`verify_kraft_wave1.py`, `verify_kraft_wave2.py`): the broken step fails on nearly half of a population in which the conclusion has no slack at all. Nor is the violation bounded — deterministic adversarial search reaches Kraft sums of 1.6875 at $n = 6$ and 1.8125 at $n = 7$, so the suprema rise with n , and no argument may assume a constant bound on the excess. The percentages over constructed populations are properties of the generators and seeds, not rates (Section 7); the shipped scripts *assert* the existence and universality statements and *print* the counts.

3.4 The strongest defence, and its refutation

Ten steelman defences were attempted under an adversarial brief and are recorded with their refutations in [21]. Two matter here. The first is the slack argument just refuted. The second is the one a sympathetic reader reconstructs first, and, we believe, what the chapter had in mind:

MAX(T) and MAX2(T) are comparator-disjoint. Every comparator of MAX(T) has both inputs on MAX-edges; q_j starts on c_j 's *lower* output, which is not a MAX-edge; a comparator with a non-MAX input is not in MAX(T); induct.

Disjointness would make (5) an exact union with nothing double-counted, and would force every q_j onto high leads throughout — hence prefix-free, hence Kraft — repairing (5) and (6) at one stroke. It fails because its premise is precisely what a pass-through violates: a pass-through lies in MAX(T) and has one blue input. On T_1 the two subnetworks share two comparators, one of them the root branch node; across the sweep they meet in 60.2 % of networks and at a branch node in 59.9 %. Disjointness *does* hold on Batcher's 8-sorter, which is why the chapter's own worked example cannot expose it. The conclusion itself survives every test we have: with no pruning at all, all 42 sorting sequences on 3 channels of length ≤ 4 and all 912 on 4 channels of length ≤ 6 satisfy it (`verify_kraft_dispute.py`).

3.5 A misstatement is not a gap

The chapter's only worked example is Batcher's odd-even 8-sorter. Reconstructed from the algorithm rather than from any table, its MAX(T) has seven comparators, seven branch nodes and *no* pass-through; its nc multiset $\{3, 3, 3, 3, 4, 4, 5\}$ matches the values printed in the chapter's Fig. 5, and $f = 96 = F(8)$ (`verify_huffman2.py`). But its MAX2 depths give $\sum_j 2^{-nc(q_j)} = 0.75 \neq 1$, so equation (6) is not even an equality in the chapter's own example. The two findings must be kept apart, since conflating them would overstate the case: 0.75 is *below* 1, step (7) needs only ≤ 1 , and the derivation goes through there unharmed, whereas T_1 gives 1.25. This also suggests how the error survived — the single example in the chapter fails the equation on the safe side, and its MAX subnetwork is clean, so it discriminates neither the literal from the tree reading of nc nor a correct premise from a false one.

4 The record: twenty entries in three inconsistent tables

4.1 No erratum, and no reader

We searched for any erratum, correction, gap note, reproof or formalization of equations (5)–(9): the OpenAlex and Google Scholar citation graphs, arXiv title and full-text search, DBLP, GitHub code and issue search, Knuth's Vol. 3 and its errata, the Stanford technical-report indexes including the CS-TR scan collection released in December 2025, Dobbelaere's table with its Wayback history, and Harder's Isabelle sources. Nothing exists, and the reason is stronger than “nobody found the error”: nobody has engaged with the argument. A search for `P(2,, MAX2, MAX subnetwork` and Kraft across some twenty-five modern papers returns zero hits, and Knuth carries no two-value exercise and never cites the chapter. There is no machine-checked version of the $P(2, N)$ bound.

4.2 Twenty entries, not one

The chapter's chain is not used at $N = 13$ only. Taking

$$S(N) \geq \max(S(N-1) + \lceil \log_2 N \rceil, S(N-2) + \lceil \log_2 F(N) \rceil), \quad S(11) = 35,$$

and regenerating $F(N)$ to $N = 32$ by exact minimization over shapes — not by the chapter's relaxation (19), which is known to agree only for $N \leq 16$ — reproduces the published column exactly at every one of $N = 13, \dots, 32$. The regeneration is cross-checked two ways: it reproduces the chapter's Table 1 for $N \leq 16$, and at $N = 13$ brute-force enumeration of all 208,012 shapes with 13 leaves agrees with the height dynamic program (`verify_novel_facts.py`).

N	$F(N)$	$\lceil \log_2 F \rceil$	one-value (proved)	eq. (8) chain	Dobbelaere [10]	Wikipedia [30]	deficit
13	392	9	43	44	44	43	1
14	424	9	47	48	48	47	1
15	480	9	51	53	53	51	2
16	512	9	55	57	57	55	2
17	784	10	60	63	63	60	3
18	848	10	65	68	68	65	3
19	960	10	70	73	73	70	3
20	1024	10	75	78	78	75	3
21	1232	11	80	84	84	—	4
22	1296	11	85	89	89	—	4
23	1408	11	90	95	95	—	5
24	1472	11	95	100	100	—	5
25	1872	11	100	106	106	—	6
26	1936	11	105	111	111	—	6
27	2048	11	110	117	117	—	7
28	2112	12	115	123	123	—	8
29	2320	12	120	129	129	—	9
30	2384	12	125	135	135	—	10
31	2496	12	130	141	141	—	11
32	2560	12	135	147	147	—	12

Three things are in that table. First, it is the first written derivation of Dobbelaere’s 2025 lower-bound column. That page carries the numbers with a one-line changelog — “2025-04-21 Tighter lower bounds for size, on suggestion of Jelmer Fiset and based on principles in [Voorh72]” — no derivation, no citation on the row (the only reference attached to the $N = 13$ row is for the depth result), and we located no publication, preprint or repository by the person credited. We can now say exactly what the column is, and we confirm it constructively rather than by absence of evidence. Second, every one of the twenty entries strictly exceeds the one-value floor, and the only route to the excess anywhere in the literature is the equation whose published derivation is invalid: the audit bears on $S(13)$ through $S(32)$. Third, the deficit column is the cost of the error, and it grows — one comparator at $N = 13$, twelve at $N = 32$.

4.3 Which column is defensible

The two most consulted public tables disagree at every N from 13 to 20. Wikipedia’s “Size, lower bound” row is exactly the one-value column, and footnotes itself “Obtained by Van Voorhis lemma and the value $S(11) = 35$ ” [30]; Dobbelaere’s is the eq. (8) column [10]; and OEIS A003075 [19], which Dobbelaere’s column header credits, lists no lower bound for any $N \geq 13$ and has no b-file. So the record is three-way inconsistent, and this work adjudicates it: the defensible column is Wikipedia’s.

The published lower-bound table for sorting-network size has no unconditional support at any $N \geq 13$. The unconditional table for $N = 13, \dots, 32$ is
43, 47, 51, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125, 130, 135.

The twenty numbers under audit are not refuted, and we make no claim that any is false; the claim is exactly and only about the status of their proof. Nor is a peer-reviewed result overturned — the value at $N = 13$ has never appeared in a peer-reviewed publication, and the best published lower bound is 43 — which lowers the stakes and correspondingly raises the value of recording the finding, since the column is propagating with no correct argument behind it.

4.4 What the table rests on

Both columns reduce to a single computed value and to no other: $S(12) = 39$ is one-value from $S(11) = 35$, and every entry at $N \geq 13$ is a chain above $S(11)$ or $S(12)$; replacing $S(11) = 35$ by 34 lowers every entry at $N \geq 12$ by at least one (`verify_novel_facts.py`). That value had exactly one machine proof, Harder’s 2.9 GB certificate [14]. A companion paper reports a second, independent one, from a rebuilt

search engine whose certificate is accepted by an unmodified extraction of the same Isabelle/HOL-verified checker [23, 22]: the deepest computational input to every published lower bound above $n = 11$ has now been confirmed twice.

5 The partial repair

The gap is the missing antichain step. We supply it for two classes, give a complementary bound at the opposite extreme, close one family of repairs structurally, and then say how much that leaves.

5.1 The exact correction of equation (5)

Lemma 4 (exact decomposition). *Let $c = \text{LCA}_B(x, y)$ and let $e(z, c)$ be the number of comparators strictly between input z and c on z 's max-path. Then $|W(x, y)| = e(x, c) + e(y, c) + g(c) + r(c)$, where $g(c)$ is the number of comparators from c to o_N inclusive on the maximum's onward path and $r(c) = nq(c) - ov(c)$ is the number the second maximum traverses after c that the maximum does not. In particular g and r depend only on c .*

The proof is three short observations, all general and all in [21]: the two values first meet at $\text{LCA}_B(x, y)$; after c their trajectories are functions of the pair of occupied leads alone; and the two pre-meeting paths are disjoint. This is the correction of equation (5), which double-counts $ov(c)$. Aggregating over the admissible pairs at a node gives the identity that replaces it.

Lemma 5 (the identity). *For every N -sorter and every branch node c ,*

$$W^*(c) = a(c) + \varepsilon(c) + \gamma(c) + r(c), \quad \varepsilon, \gamma, r \geq 0,$$

where $\varepsilon(c)$ measures how far the maximum's pre-meeting paths exceed the branch-tree heights and $\gamma(c)$ is exactly the number of pass-throughs on the maximum's stem from c to o_N . Hence $p(2, T) = \max_c [a(c) + \varepsilon(c) + \gamma(c) + r(c)]$.

This is an exact expression for $p(2, T)$ in branch-tree data plus named non-negative corrections, verified with zero violations over the 32,043-sorter corpus of `verify_kraft_wave2.py`. One consequence deserves separate statement: equation (8) is *equivalent* to

$$\sum_c 2^{a(c)} \leq 2^{\max_c [a(c) + \varepsilon(c) + \gamma(c) + r(c)]},$$

a statement about a binary tree decorated with three non-negative integer functions, and it is *false* for arbitrary decorations, since $\varepsilon = \gamma = r = 0$ refutes it on any tree with two or more internal nodes. The entire content of equation (8) is therefore a realizability constraint — which decorated trees arise from an actual sorter — and that is the shape of the missing lemma.

5.2 The escape-free case

Theorem 6 (equation (8) for escape-free sorters). *For every escape-free N -sorter T with branch tree B ,*

$$1 \geq \sum_c 2^{-nq(c)} \geq \sum_c 2^{-(p(2, T) - a(c))} = 2^{-p(2, T)} f(B),$$

hence $p(2, T) \geq \lceil \log_2 f(B) \rceil$ and $|T| \geq S(N - 2) + \lceil \log_2 f(B) \rceil$.

The proof is elementary and is in [21], its lemmas checked by `verify_kraft_wave1.py` and `verify_kraft_wave2.py`. The two steps the chapter omits are these. In the colouring of Definition 1, every low output lead is blue, a high output lead is red exactly when its comparator has a red input, and a comparator is a branch node exactly when both its inputs are red. In an escape-free network the second maximum therefore traverses *no branch node* after the meeting, so the $N - 1$ sources — the low output leads of the branch nodes — are pairwise non-ancestral under the second maximum's lead successor map. That is the antichain, and Kraft's inequality applies; the per-node inequality $nq(c) \leq p(2, T) - a(c)$

is Lemma 4 with the deepest-leaf pairing. Two things change relative to the chapter, both essential: equation (6) becomes an *inequality*, which is why Batcher’s own 8-sorter gives 0.75 and the argument is unharmed, and the antichain is *proved* rather than assumed. In an arbitrary sorter the lead-level successor map is not even a function, which is the precise sense in which $\text{MAX2}(T)$ is not a tree.

5.3 Isolating the hypotheses

Escape-freeness enters Theorem 6 only through (C) and (D) of Definition 2. Isolating them widens the class.

Theorem 7 (equation (8) under (C) and (D)). *If an N -sorter satisfies (C) and (D), then $p(2, T) \geq \lceil \log_2 f(B) \rceil$.*

Proof: verbatim that of Theorem 6, which uses escape-freeness nowhere else. The gain is small and we say so. On the 32,043-sorter corpus (C) $\wedge$ (D) holds on 7,214 networks against 7,167 escape-free ones, a widening of 0.7%; over the exhaustive $n = 5$ universe it is 33,030 against 32,850, a widening of 0.55%; and at $n = 4$, in both universes, (C) and (D) are disjoint on escaping sorters, so the conjunction buys nothing at all. Equation (8) holds on all 7,214. The value of Theorem 7 is not its coverage but that it names the two things, and only the two things, a general proof must supply.

5.4 The pass-through-rich end

Proposition 8 (an upper bound on $f(B)$). *For every N -sorter, $f(B) \leq 2^{p(2, T)} \sum_c 2^{-\gamma(c)}$. In particular equation (8) holds whenever $\sum_c 2^{-\gamma(c)} \leq 1$, and whenever the root’s stem carries at least one pass-through and every edge of B carries at least two.*

Unlike (C) and (D), this hypothesis is computable from the MAX subnetwork alone, and the class is non-empty and not contained in the escape-free class. **But it must be quoted with its caveat.** Appending inert comparators raises $p(2, T)$ while leaving $f(B)$ fixed, so the sorters it reaches are exactly those with large slack — the witnesses have $p(2, T) = 7$ against a bound of 5. It is recorded because it completes the picture from the opposite direction, not because it is useful for size-optimal networks.

5.5 Coverage: where the open problem lives

This is the most useful new fact here, and the most uncomfortable. Over the 32,043-sorter corpus of `verify_kraft_wave2.py` — exhaustive at $n = 4$ in both universes and at $n = 5$ in the minimal universe, plus constructed families to $n = 11$, inert-augmented families and random $n = 6, 7$ sorters — each condition fires as follows.

sufficient condition	count	fraction	kind
clean	5,891	0.1838	structural
escape-free (Theorem 6)	7,167	0.2237	structural
hypothesis of Proposition 8	0	0.0000	structural
structural union	7,167	0.2237	
(C) $\wedge$ (D) (Theorem 7)	7,214	0.2251	computed
$\sum_c 2^{-\gamma(c)} \leq 1$	18	0.0006	computed
$\sum_c 2^{-r(c)} \leq 1$	16,052	0.5010	computed
union of all six	16,053	0.5010	

Equation (8) fails on none of the 32,043. Structural and computed conditions must not be blurred: escape-freeness and the hypothesis of Proposition 8 are properties of a network’s shape, whereas (C), (D) and the two sums are numbers one computes per network, and a per-network computation is not a theorem. The proved classes cover 22.4% of the corpus structurally, and 49.9% **of it has no proof of equation (8) by any route in this work**. Proposition 8’s hypothesis fires on no network here at all: its witnesses have to be built by inert padding, which is exactly the slack regime its caveat describes.

The uncovered zone is *intermediate* in pass-through count. On this corpus the fraction with no covering condition is 0.000 at zero pass-throughs — all 5,891 such networks are clean, hence covered — and then 0.346, 0.740 and 0.995 at one, two and three; beyond three the corpus thins to a few dozen networks per bucket and the fraction falls back irregularly, reaching zero only at the largest counts present. Equation (8) is provable by Theorem 6 at the pass-through-poor end and by Proposition 8 at the pass-through-rich end, and the open problem lives strictly in between.

5.6 What is closed off

Proposition 9 (per-node charging is impossible). *On any T with $f(B) = 2^{p(2,T)}$ one has $\sum_c 2^{a(c)-p(2,T)} = 1$ exactly. Hence for any per-node exponent $X(c) \leq p(2,T)$, $\sum_c 2^{a(c)-X(c)} \geq 1$, with equality if and only if $X(c) = p(2,T)$ at every branch node — and tight sorters with non-constant $W^*(c)$ exist. No certificate built from each node’s own pruning can prove equation (8).*

The witness is an 18-comparator 7-sorter with $f = 2^{p(2,T)} = 128$ and $W^* = \{6, 7, 7, 7, 7, 7\}$; the strongest member of the family, charging each node against its own $W^*(c)$, is refuted outright by an adversarially found 15-comparator 6-sorter with $\sum_c 2^{a(c)-W^*(c)} = 33/32$. Four further repair families are closed with explicit counterexamples in [21]. What remains is worth stating plainly: without the Kraft step the chain yields only $p(2,T) \geq \max_j \text{nc}(c_j)$, which for the f -minimizing 13-leaf trees is 42. The Kraft step carries the entire excess over the one-value floor, and the Kraft step is the broken one.

5.7 One thing the repair does buy

Theorem 6 is per-network and un-minimized, over the network’s own branch tree — exactly the hypothesis a shape-based case analysis of the $n = 13$ problem has to assume outright. Two machine checks close the identification: $f(B)$ equals the shape-theoretic $V(T)$ on all 208,012 plane shapes with 13 leaves and on every shape with at most 11 leaves, and at $N = 13$ and the tabulated size Theorem 6 forces $f(B) \leq 512$, which is that analysis’s admissibility condition (`verify_novel_facts.py`, `verify_huffman2.py`).

Corollary 10 (the shape case split, unconditional on the escape-free class). *Let C be an escape-free 13-sorter of the tabulated size. Then its max branch tree is one of exactly 84 admissible plane shapes (6 up to reflection), its root splits the 13 leaves as $5 + 8$, $6 + 7$ or $4 + 9$, and its leaf-depth profile is one of exactly four. If C and its reverse-and-flip dual are both escape-free, both branch trees are so constrained.*

The finite shape enumeration and the identification $f = V$ are machine-checked; Corollary 10 inherits from Theorem 6 the status of a human proof. What changes is that the case split no longer needs the chapter: it is unconditional on a class we can define, check and measure.

6 What is not disturbed

6.1 The one-value bound, and Harder’s Lemma 17

A reader who checks Harder’s paper will find the same identification we attack. His Lemma 17 reproduces the classical one-value bound with a full proof, and in it:

“if we take the union of the pruned paths for all i and remove the common output n , we obtain a binary tree rooted in the comparator gate connected to output n , where the leaves are all inputs of c and all inner vertices are comparator gates ... the leaf for every input i has a depth of $\delta(c, i)$.” — [13], proof of Lemma 17

Identifying max-path comparators with branch nodes is exactly the p. 121 premise, and by Proposition 3 it is false whenever pass-throughs exist: $\delta(c, i)$ is a literal comparator count and can exceed the branch-tree depth. **It is harmless there**, for a one-line reason. The one-value bound needs only $\max_i \delta(c, i) \geq \max_i \text{depth}_B(i) \geq \lceil \log_2 n \rceil$, and literal depth *dominates* branch depth, so the inequality goes the safe way. $S(11) = 35$, $S(12) = 39$ and the published $S(13) \geq 43$ stand. We record this because a reader who notices Harder’s phrasing will otherwise ask whether the audit also breaks $S(12)$, and because it strengthens the account of how the error survived: the same imprecision was reproduced,

unnoticed, in a 2020 paper that is the current authority for $S(11)$ and $S(12)$. One smaller point. Harder’s Lemma 17 is a paper proof — his Isabelle development verifies the certificate checker, not Van Voorhis’s bound — so $S(12) = 39$ is not machine-checked either.

6.2 What is still known about a minimum-size 13-sorter

The following is unconditional. It uses only that deleting a channel at either polarity from an N -sorter removes exactly the comparators on that channel’s extremal path and leaves an $(N - 1)$ -sorter, so $|C| \geq S(N - 1) + \delta(C, i)$; that $\delta(C, i) = \text{depth}_B(i) + pt(C, i)$, which is Proposition 3 used in the safe direction; Kraft equality on a full binary tree; and the duals for the minimum. No step uses equation (8). Write m for the tabulated size.

Proposition 11 (unconditional structure). *Let C be a 13-sorter with $|C| = m$. Then: (i) every extremal path of C , at either polarity, has at most $m - S(12) = 5$ comparators, so $\text{depth}_B(i) + pt(C, i) \leq 5$ for every channel i ; (ii) a leaf at branch-depth 5 has a pass-through-free max-path; (iii) both branch trees have height in $\{4, 5\}$, and the leaf-depth multiset of either is one of exactly 13 Kraft-tight profiles with all depths at most 5; (iv) if any extremal path has length 5, deleting that channel leaves a size-optimal 12-sorter in which both branch trees have height exactly 4; (v) C has a one-channel deletion minor that is a 12-sorter with 39 or 40 comparators and a two-channel deletion minor that is an 11-sorter with 35 or 36, and these are the only two places in the deletion chain where the slack is 1; and (vi) $|\text{maxpath}(i) \cup \text{minpath}(j)| \leq m - S(11) = 9$ for all $i \neq j$.*

Item (ii) is the point of contact: it links pass-throughs, the audit’s central object, to the $n = 13$ question directly and without appeal to the disputed inequality. Item (vi) is the max/min analogue of the chapter’s max/second-max quantity, and it is worth a sentence because of what it avoids. The chapter’s two-channel deletion removes the maximum and the *second* maximum, and its Kraft step needs $\text{MAX2}(T)$ to be a binary tree, which is the false premise; the max/min deletion removes the maximum and the *minimum*, and both objects it involves are honest Kraft-tight binary trees with N leaves. Whether $\min_C \max_{i \neq j} |\text{maxpath}_C(i) \cup \text{minpath}_C(j)|$ admits a Kraft-style lower bound we do not know — the two paths can share comparators — nor have we established that the variant is new.

7 What we trust

Following Harder [13], we enumerate the trusted base rather than claim correctness. We do not require, or expect, it to be free of bugs; only that no bug was triggered in the runs reported here.

component	trusted base?	note
hardware, operating system, Python 3.14	yes	general-purpose
the two audit verifiers of Section 3	yes	mutually independent, no shared code; the counterexample needs neither and is checkable by hand they import the audit verifier’s primitives, so they are extensions, not independent re-derivations human proofs; no proof assistant has checked them
the two wave verifiers of Section 5	yes	
Theorems 6, 7; Lemmas 4, 5; Propositions 8, 9, 11	yes	
the identification $f = V$ and the 13-leaf shape enumeration	yes	
the prevalence and coverage percentages	yes — scoped	properties of the named corpora and seeds, not rates
Isabelle/HOL kernel and extraction; GHC	yes	for $S(11) = 35$ only
the search engines and certificate generators behind $S(11) = 35$	no	the value is re-derived by the verified checker [14, 22, 23]
the two archival items [25, 28]	yes — not obtained	see below

Five things follow, and none of them is a formality.

Independence is not uniform. The two audit scripts share no code, so the counterexample and the prevalence figures are re-derived twice from scratch. But the two wave verifiers that carry Section 5 *import* the first script’s primitives; they are extensions, not independent re-derivations, and should be read as such.

We do not meet the de Bruijn criterion for the new mathematics. A computer-assisted proof should be checkable by an independent small program. The certified value $S(11) = 35$ meets that criterion. Theorems 6 and 7 do not: they are human proofs whose lemmas are tested on samples, and Proposition 9’s own history — 900 samples with supremum exactly 1.000000, followed immediately by an adversarial counterexample — is why we do not offer sample agreement as more than weak evidence. Formalizing Theorem 6 is a small task and is the natural next step; it would also make Corollary 10 fully machine-checked.

Every percentage here is a corpus property, not a rate. The 61.0%, 44.4% and 27.1% of Section 3 are properties of a fixed generator set and seed, and the coverage fractions are properties of the 32,043-sorter corpus. Both are regenerated by the named scripts, and both establish that the phenomena are common in ordinary networks rather than that they occur at any rate under a measure on sorting networks. The two exhaustive populations — all 708 minimal 4-sorters and all 149,040 minimal 5-sorters — are the only figures here that are rates, and they are rates only at those two widths. We do not know what fraction of *size-optimal* networks at large n is escape-free, and a reader is entitled to ask.

Two archival items were not obtained, and one may be the origin of the error. The *Transactions* note [28] is what the chapter cites specifically for the false structural claim. Three independent readers of it — Knuth, Parberry [20] and Harder — report only the one-value bound, and Harder’s rendition contains the same identification, which makes it likely the false premise originates there; if the note states the claim correctly, the chapter’s error is introduced rather than inherited. The 1971 Stanford dissertation [25] is not digitized and is absent from the Stanford CS-TR scan collection released in December 2025, which we checked; only the divide-sort-merge report [26] is there. Obtaining either is procurement, not research, and neither affects $S(13) \geq 43$.

The status of the disputed inequality. Equation (8) is neither proved nor refuted. It has substantial empirical support — zero violations across the constructed and exhaustive corpora above, roughly 15,000 adversarially searched networks aimed directly at it [21], and consistency with every exact $S(n)$ known, the slacks $S(N) - S(N - 2) - \lceil \log_2 F(N) \rceil$ for $N = 3, \dots, 12$ being 0, 0, 0, 1, 0, 0, 1, 2, 2, 1, so a single negative entry would have refuted it. A full repair must be global, must *handle* pass-throughs rather than exclude them, must survive the tight regime, and must reach $\lceil \log_2 f(B) \rceil$ rather than $\max_j \text{nc}(c_j)$. Failing that, the honest state of the record is $43 \leq S(13) \leq 45$, and correspondingly at every N up to 32. The audit was carried out by language-model agents under a fixed protocol, one reading the primary source and a second instructed to defend the chapter and refute the first; no claim here rests on a model’s assertion, since every quantitative statement is produced by one of the five scripts and the central counterexample is checkable on paper.

Van Voorhis closed his chapter by predicting that the next improvement in the lower bound for $S(N)$ would come from an approximation of $P(3, N)$. Fifty-four years on, $P(2, N)$ is still open.

References

- [1] K. E. Batchier. Sorting networks and their applications. *AFIPS Spring Joint Computer Conference*, 1968, 307–314.
- [2] R. C. Bose, R. J. Nelson. A sorting problem. *J. ACM* 9(2), 1962, 282–296.
- [3] D. Bundala, J. Závodný. Optimal sorting networks. *LATA 2014*, LNCS 8370, 236–247. arXiv:1310.6271.
- [4] R. Burca, M. Raschip. Finding minimal-size sorting networks using deep Q-learning. *ICAART 2026*.
- [5] M. Codish, L. Cruz-Filipe, M. Frank, P. Schneider-Kamp. Sorting nine inputs requires twenty-five comparisons. *J. Computer and System Sciences* 82(3), 2016, 551–563. Conference version ICTAI 2014; arXiv:1405.5754.
- [6] M. Codish, L. Cruz-Filipe, P. Schneider-Kamp. The quest for optimal sorting networks: efficient generation of two-layer prefixes. *SYNASC 2014*. arXiv:1404.0948.

- [7] L. Cruz-Filipe, K. F. Larsen, P. Schneider-Kamp. Formally proving size optimality of sorting networks. *J. Automated Reasoning* 59, 2017, 425–454.
- [8] L. Cruz-Filipe, P. Schneider-Kamp. Formalizing size-optimal sorting networks: extracting a certified proof checker. *ITP 2015*. arXiv:1502.05209.
- [9] L. Cruz-Filipe, P. Schneider-Kamp. Minimizing sorting networks at the sub-comparator level. *LPAR 2024*, EPiC Series in Computing.
- [10] B. Dobbelaere. *Smallest and fastest sorting networks for a given number of inputs*. https://bertdobbelaere.github.io/sorting_networks.html; changelog entry of 2025-04-21; consulted 2026-08-22.
- [11] T. Ehlers, M. Müller. Faster sorting networks for 17, 19 and 20 inputs. arXiv:1410.2736, 2014.
- [12] R. W. Floyd, D. E. Knuth. The Bose–Nelson sorting problem. In *A Survey of Combinatorial Theory*, North-Holland, 1973, 163–172.
- [13] J. Harder. *An answer to the Bose–Nelson sorting problem for eleven and twelve elements*. arXiv:2012.04400v3, 2022.
- [14] J. Harder. *Certificate for the minimal size of 11- and 12-channel sorting networks*. Zenodo, doi:10.5281/zenodo.4108365; software `jix/sortnetopt`, doi:10.5281/zenodo.4139152.
- [15] H. Juillé. Evolution of non-deterministic incremental algorithms as a new approach for search in state spaces. *ICGA 1995*, 351–358.
- [16] N. Kahale, T. Leighton, Y. Ma, C. G. Plaxton, T. Suel, E. Szemerédi. Lower bounds for sorting networks. *STOC 1995*, 344–353.
- [17] D. E. Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley, 1973. Section 5.3.4, exercise 42 and its answer.
- [18] V. Marinov, D. Gregg. Sorting networks: the final countdown. arXiv:1502.05983, 2015.
- [19] OEIS Foundation Inc. Sequence A003075: minimal number of comparisons needed for a sorting network on n elements. *The On-Line Encyclopedia of Integer Sequences*, consulted 2026-08-22.
- [20] I. Parberry. A computer-assisted optimal depth lower bound for nine-input sorting networks. *Supercomputing 1989*, 152–161.
- [21] The authors. *Artifact repository: verifiers, patches, theory documents, evidence manifests*. To be deposited. Cited throughout for check-level indexes, full proofs, refuted routes and measurement records.
- [22] The authors. *Certificate of the minimal size of 11-channel sorting networks, independently regenerated*. 2,442,317,348 bytes, SHA-256 672c433f . . . To be deposited.
- [23] The authors. *A cutoff theorem for exhaustive sorting-network search, and an independently certified $S(11) = 35$* . Companion paper.
- [24] A. Tozawa, K. Sadakane. Odd-even transposition sort is an optimal stable standard sorting network. *Information Processing Letters* 195, 2026, article 106659.
- [25] D. C. Van Voorhis. *Efficient sorting networks*. Ph.D. dissertation, Stanford University, 1971. Not obtained.
- [26] D. C. Van Voorhis. *A lower bound for sorting networks that use the divide-sort-merge strategy*. Stanford Digital Systems Laboratory TR 17, STAN-CS-71-238, 1971.
- [27] D. C. Van Voorhis. Toward a lower bound for sorting networks. In R. E. Miller, J. W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, 1972, 119–129. Equation numbers, Theorem 1, Table 1 and Figs. 1–5 cited here are the chapter’s own; page numbers are book pages.
- [28] D. C. Van Voorhis. An improved lower bound for sorting networks. *IEEE Transactions on Computers* C-21(6), 1972, 612–613. Not obtained.
- [29] A. Vlad, B. Pascu, C. Cioata, M. Raschip. Identifying optimal-size sorting networks with reinforcement learning. *ICTAI 2025*.
- [30] *Sorting network*. Wikipedia, revision of 2026-08-15; consulted 2026-08-22.